

COVID-19 CT Multi-class Segmentation: U-Net vs. DeepLabV3+ with Optuna Hyperparameter Search

Nick Kantiotis

MSc Artificial Intelligence & Visual Computing (AIVC)

June 24, 2026

Abstract

We address pixel-wise semantic segmentation of chest CT scans into four classes — *background*, *left lung*, *right lung* and *infection* — on the public COVID-19 CT Lung and Infection dataset (20 annotated volumes, Zenodo 3757476). Two encoder-decoder architectures, **U-Net** and **DeepLabV3+**, are trained on 2D axial slices with a combined Dice + Cross-Entropy loss, and tuned with **Optuna** (5 trials each, maximising validation foreground Dice). On a held-out, patient-level test split, **U-Net** attains a mean foreground Dice of **0.915** (IoU 0.854) versus **0.858** (IoU 0.782) for DeepLabV3+. The gap is driven almost entirely by the small, imbalanced *infection* class (Dice 0.794 vs. 0.635), where DeepLabV3+ under-segments lesions (sensitivity 0.50 vs. 0.77).

Contents

1	Introduction	1
2	Dataset	1
3	Methodology	2
4	Models	2
5	Hyperparameter Search (Optuna)	2
6	Results	4
6.1	Training curves (best trial)	5
6.2	Confusion matrices	6
6.3	Qualitative overlays	7
6.4	Per-trial training curves	9
7	Conclusions and Limitations	9

1 Introduction

The COVID-19 CT Lung and Infection Segmentation dataset [1] contains **20 labelled CT volumes** (NIfTI `.nii.gz`) from the Coronacases Initiative and Radiopaedia, annotated by radiologists into left lung, right lung and infection. The task is to segment each axial slice into these structures, supporting automated assessment of COVID-19 lung involvement. We compare two widely used segmentation networks under an identical training and tuning protocol to isolate the effect of architecture, with particular attention to the clinically critical but hardest target: the infection (lesion) class.

2 Dataset

The dataset provides 20 three-dimensional CT volumes with dense voxel labels over four classes (Table 1). Annotations are released under the **CC BY-NC-SA** licence [1].

Table 1: Label encoding of the four segmentation classes.

Index	Class
0	background
1	left lung
2	right lung
3	infection (lesion)

Patient-level split. The 20 volumes are split deterministically (seed 42) into **14 train / 3 validation / 3 test** patients. Crucially, slices from a single volume never cross splits, preventing data leakage and keeping the evaluation honest despite the small number of subjects.

3 Methodology

2D slice segmentation. Each 3D volume is sliced along the axial axis, producing thousands of 2D samples from only 20 volumes. Background-only slices (fewer than 50 foreground pixels) are discarded.

Preprocessing. Each slice is clipped to the lung Hounsfield-Unit window $[-1250, 250]$, min-max normalised to $[0, 1]$, resized to 256×256 , and replicated to 3 channels to match ImageNet-pretrained encoders.

Loss. We optimise a sum of multiclass **Dice loss** and **Cross-Entropy**, which together handle the strong class imbalance (the infection class occupies a tiny fraction of pixels).

Optimisation. Adam optimiser with a `ReduceLROnPlateau` schedule (monitoring validation Dice) and early stopping (patience 10 epochs). Data augmentation (Albumentations: flips, shift/scale/rotate, brightness/contrast) is applied at a strength selected per trial during tuning.

4 Models

Both architectures are built with `segmentation_models_pytorch`, take a $3 \times 256 \times 256$ input and output 4-channel per-class logits.

- **U-Net** — symmetric encoder-decoder with skip connections that recover spatial detail lost in downsampling. (≈ 14.3 M parameters with a ResNet18 encoder.)

- **DeepLabV3+** — atrous spatial pyramid pooling (ASPP) for multi-scale context aggregation. (≈ 12.3 M parameters with a ResNet18 encoder.)

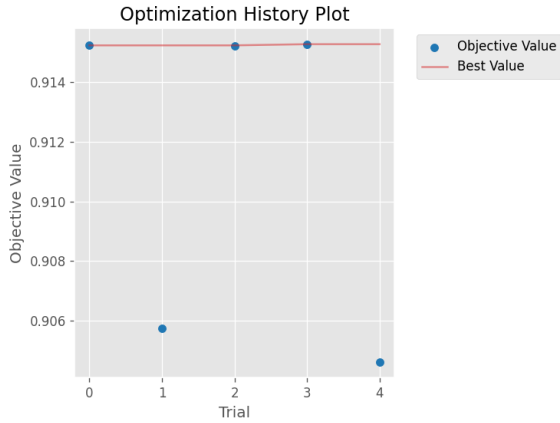
The encoder is a tunable hyperparameter (`resnet18` / `resnet34` / `efficientnet-b0`), initialised with ImageNet weights. Optuna selected `resnet18` as the best encoder for both models.

5 Hyperparameter Search (Optuna)

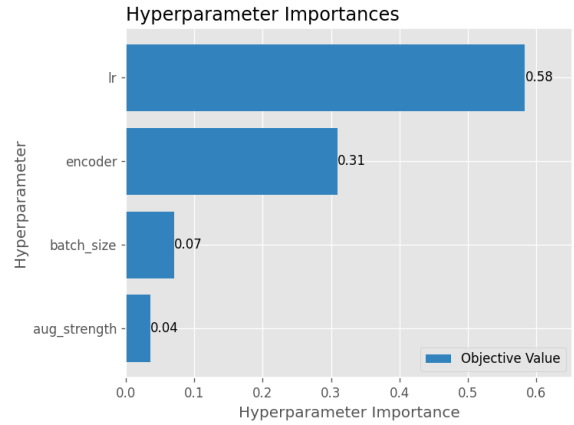
Hyperparameters are tuned with **Optuna** [4] using a TPE sampler (seed 42) and a **MedianPruner** (5 warm-up steps) to terminate unpromising trials. We run **5 trials per model**, each up to 50 epochs, maximising the validation mean foreground Dice. The search space spans learning rate (10^{-5} – 10^{-2} , log scale), batch size $\{4, 8, 16\}$, encoder $\{\text{resnet18}, \text{resnet34}, \text{efficientnet-b0}\}$ and augmentation strength $\{\text{none}, \text{light}, \text{strong}\}$. The best configurations are reported in Table 2.

Table 2: Best hyperparameters per model (Optuna).

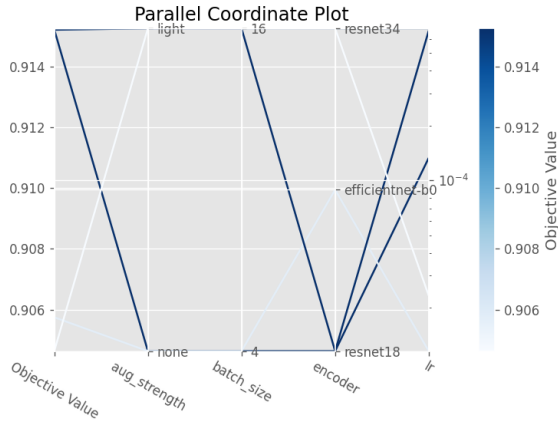
Model	lr	batch size	encoder	aug. strength
U-Net	6.65×10^{-4}	16	resnet18	light
DeepLabV3+	1.33×10^{-4}	4	resnet18	none



(a) Optimisation history



(b) Parameter importance



(c) Parallel coordinate



(d) Slice plot

Figure 1: Optuna study visualisations for U-Net.

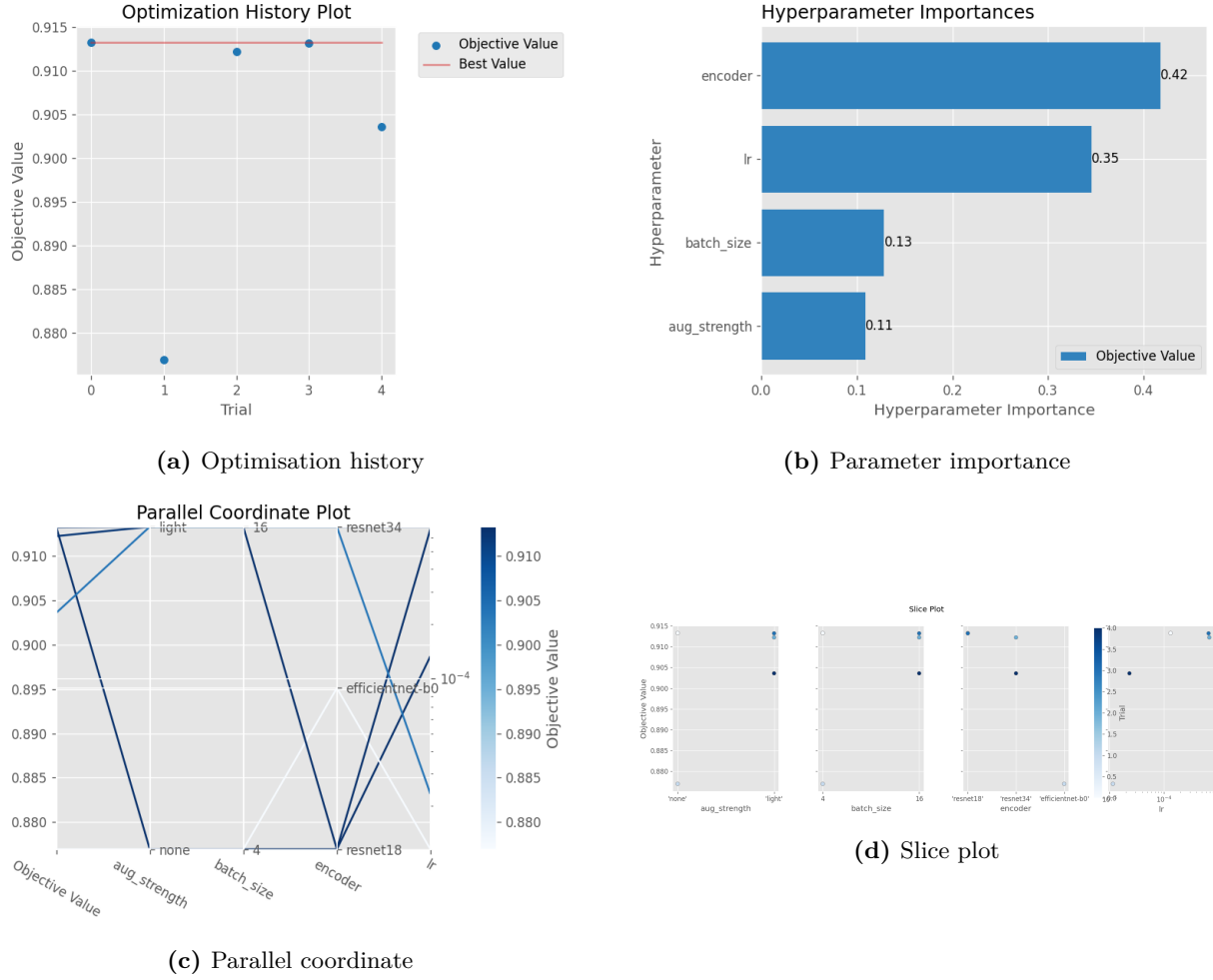


Figure 2: Optuna study visualisations for **DeepLabV3+**.

6 Results

All metrics are computed on the held-out **test split** (3 patients, unseen during training and tuning).

Table 3: Test-set comparison. Best per column in **bold**.

Model	Dice left	Dice right	Dice infection	mean FG Dice	mean FG IoU	Params (M)
U-Net	0.980	0.971	0.794	0.915	0.854	14.3
DeepLabV3+	0.978	0.960	0.635	0.858	0.782	12.3

U-Net wins overall, and the gap is driven almost entirely by the *infection* class (0.794 vs. 0.635 Dice) — the hardest, smallest and most imbalanced structure. Per-class metrics are detailed in Tables 4 and 5.

Table 4: U-Net per-class metrics.

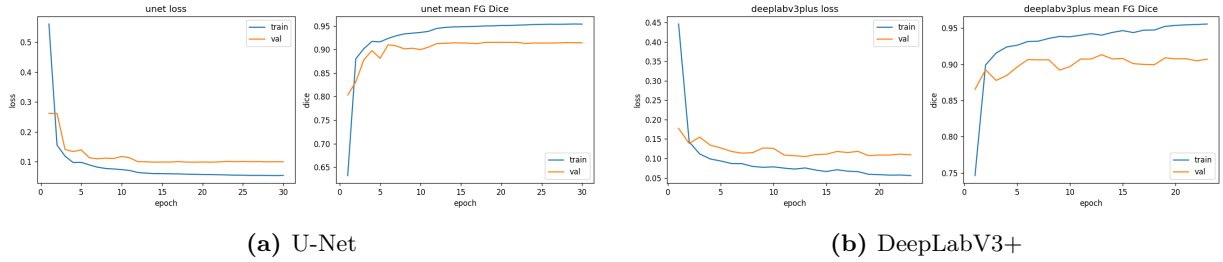
Class	Dice	IoU	Sensitivity	Specificity
background	0.998	0.996	0.998	0.986
left_lung	0.980	0.960	0.976	0.999
right_lung	0.971	0.944	0.982	0.998
infection	0.794	0.659	0.770	0.999

Table 5: DeepLabV3+ per-class metrics.

Class	Dice	IoU	Sensitivity	Specificity
background	0.998	0.995	0.998	0.979
left_lung	0.978	0.957	0.981	0.999
right_lung	0.960	0.923	0.978	0.997
infection	0.635	0.465	0.500	1.000

DeepLabV3+ reaches an infection **sensitivity of only 0.50** (vs. U-Net’s 0.77) while its specificity stays at 1.0 — i.e. it under-segments the small lesions, missing half of the infection pixels.

6.1 Training curves (best trial)

**Figure 3:** Loss and mean foreground Dice vs. epoch for the best trial of each model.

6.2 Confusion matrices

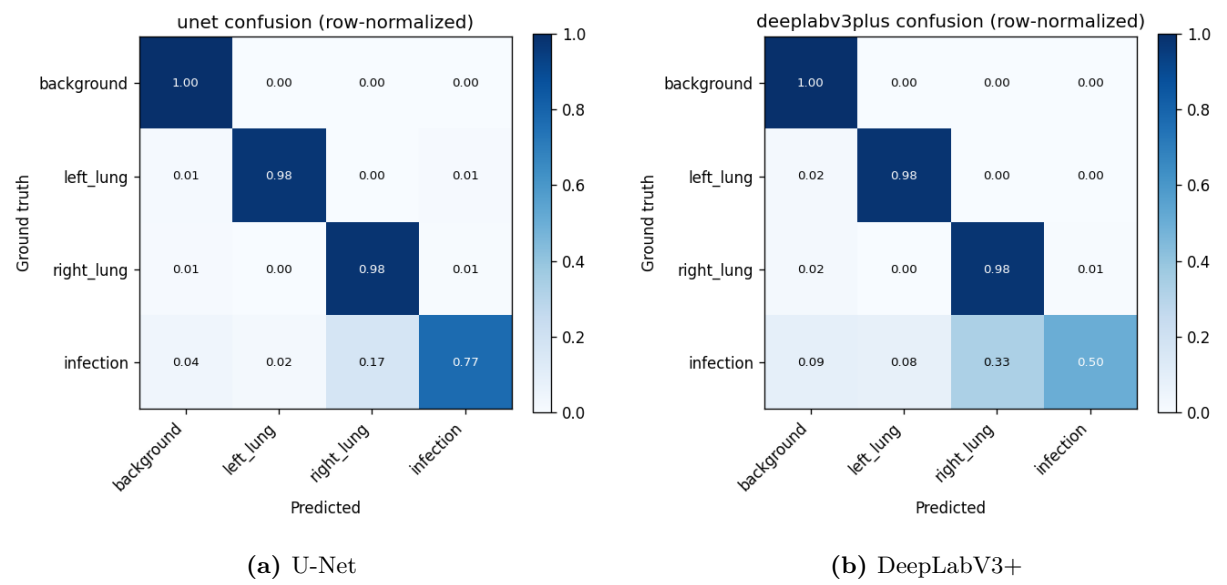


Figure 4: Pixel-level, row-normalised confusion matrices over the four classes.

6.3 Qualitative overlays

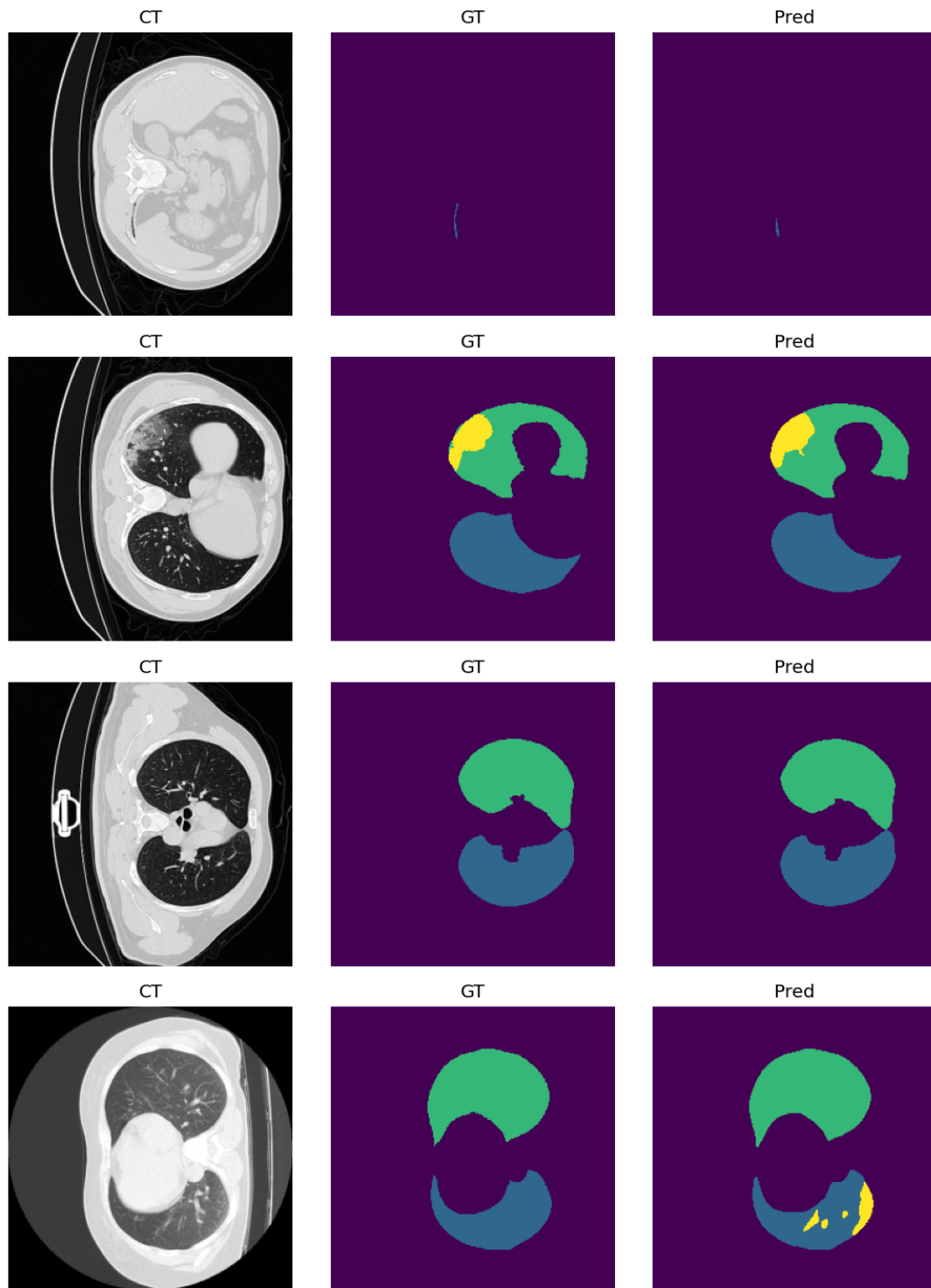


Figure 5: U-Net qualitative results on test slices: CT / ground truth / prediction.

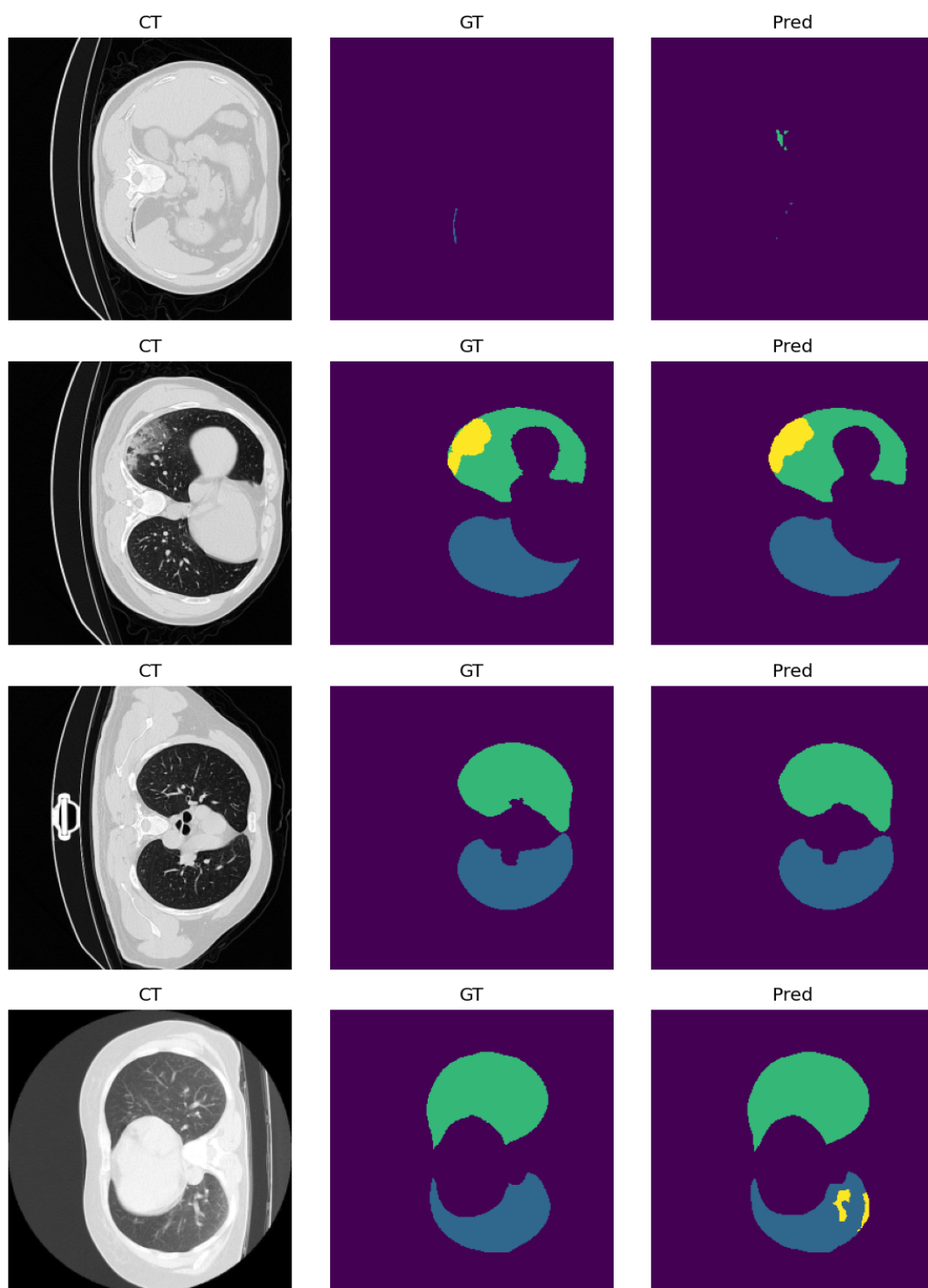


Figure 6: DeepLabV3+ qualitative results on test slices: CT / ground truth / prediction.

6.4 Per-trial training curves

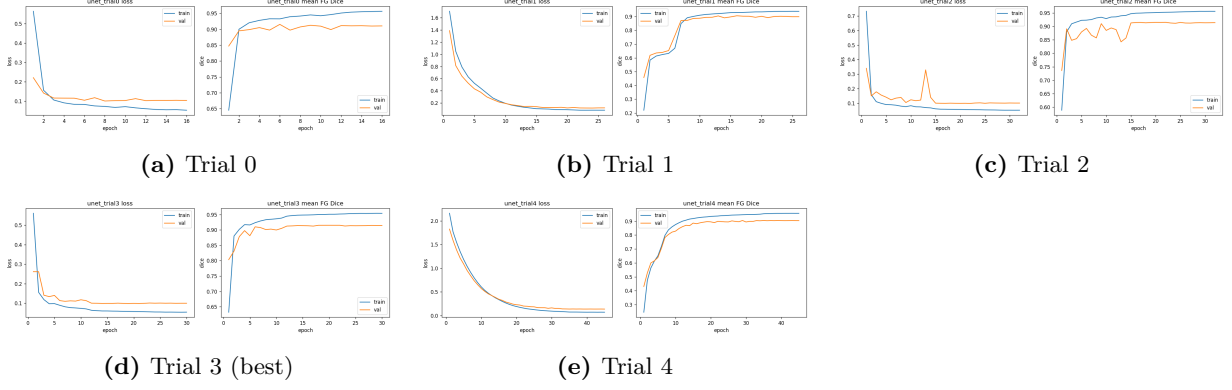


Figure 7: U-Net per-trial training curves (Optuna).

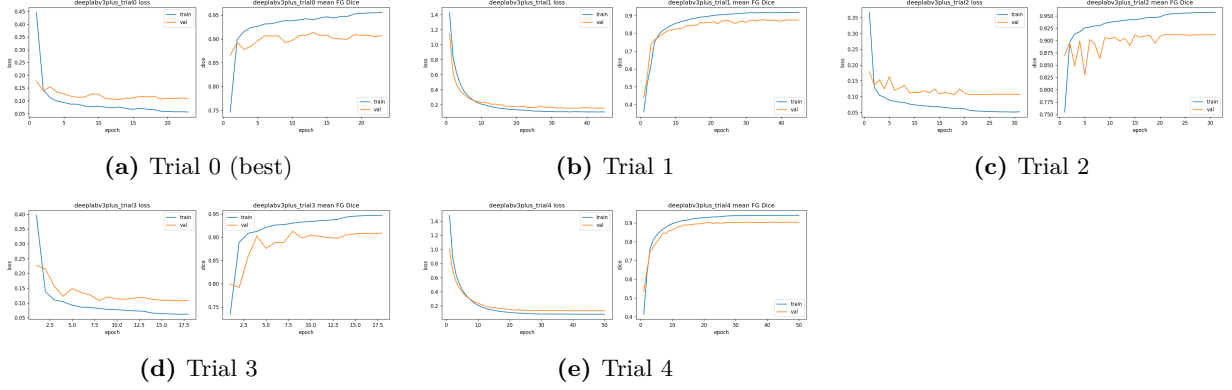


Figure 8: DeepLabV3+ per-trial training curves (Optuna).

7 Conclusions and Limitations

- **U-Net is the stronger model here**, primarily because it segments the small, scattered infection regions far better (Dice 0.79 vs. 0.64, sensitivity 0.77 vs. 0.50).
- **Lung segmentation is essentially solved** by both architectures (Dice $\approx$ 0.97–0.98) — a large, high-contrast target.
- **Infection is the bottleneck**: tiny, low-contrast and heavily imbalanced. The Dice+CE loss and augmentation help but do not close the gap.
- **Small dataset (20 volumes)**. Patient-level splitting keeps evaluation honest, but generalisation is limited; more volumes, 3D context, and lesion-focused losses are the natural next steps.

References

- [1] J. Ma *et al.*, “Towards Data-Efficient Learning: A Benchmark for COVID-19 CT Lung and Infection Segmentation,” Zenodo 3757476, 2020. <https://zenodo.org/records/3757476>. Licence: CC BY-NC-SA.

- [2] O. Ronneberger, P. Fischer, T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” MICCAI, 2015.
- [3] L.-C. Chen *et al.*, “Encoder–Decoder with Atrous Separable Convolution for Semantic Image Segmentation (DeepLabV3+),” ECCV, 2018.
- [4] T. Akiba *et al.*, “Optuna: A Next-generation Hyperparameter Optimization Framework,” KDD, 2019.
- [5] P. Iakubovskii, “Segmentation Models PyTorch,” https://github.com/qubvel/segmentation_models.pytorch, 2019.